pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Exploratory Data Analysis

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The problem: a file and its cover note

The cover note from the operations team, as of 2026-09-30

200
bonds

489,000,000
dollars of face

490,819,215
dollars of market value

Control totals

Numbers that come from outside the file, which the file should add up to. A file that does not match its control totals is not ready to use.

```
raw = pd.read_csv(data_folder + 'holdings_messy.csv')
expected_bonds = 200
expected_face = 489_000_000
expected_value = 490_819_215
```

raw is the file exactly as it arrived. Nothing today changes it.

The portfolio manager asks for

The market value of the book

Its average OAS

Its average yield

Three rules for the day

The file as received is never changed.

Every fix is a decision.

Every fix shows its effect.

The file is called holdings_messy.csv. The name is fair warning.

The EDA routine: know the table before relying on it

1 Sanity checks	2 Missing values	3 One column at a time	4 Outliers	5 Two columns at a time	6 Report
Right size and types? Any repeats, untidy text, impossible values?	How many are missing, where, and what to do about them?	The range and the shape of each column: univariate analysis.	Values far from the rest. Is each an error or a real bond?	How columns move together: bivariate analysis.	Each finding, the decision, and its effect on the numbers.
`duplicated()`	`isna().sum()`	`describe(), skew()`	`quantile()`	`corr()`	`headline()`

The file does not agree with the note

The number	Cover note	File as received
Rows	200	204
Market value, dollars	490,819,215	497,769,925

As received, the mean OAS is 187.83 bp and the mean yield 6.051 percent. These would go into the pack if nobody looked.

```
def headline(label, table):
    print(label)
    print('   rows:', len(table))
    print('   market value:', table['market_value'].sum())
    print('   mean OAS:', round(table['oas'].mean(), 2))
headline('As received', raw)   # before and after every fix
```

In Excel: `=COUNTA(B2:B205)` returns 204, with the CUSIP in column B

Sanity checks: questions with a known answer

The check	pandas	Excel
Size and totals	`len(raw), raw['par_held'].sum()`	`=COUNTA(B2:B205), =SUM(R2:R205)`
Data types	`raw.dtypes`	`=ISNUMBER(L2), =ISTEXT(L2)`
Text to a date	`pd.to_datetime(column, format='%Y-%m-%d')`	`done by Excel as the CSV opens: =ISNUMBER(J2)`
Repeated rows	`book.duplicated(), book['cusip'].duplicated()`	`=COUNTIF($B$2:$B$205, B2) > 1`
Identifiers	`book['ticker'].str.strip().str.upper()`	`=LEN(B2), =UPPER(TRIM(C2))`
Categories	`book['rating'].isin(ratings['rating'])`	`=COUNTIF(ratings!A:A, F2) > 0`
Impossible values	`(book['price'] <= 0).sum()`	`Highlight Cells Rules, Less Than, 0`
Price dates	`book['price_date'] < book['as_of_date']`	`Data, Filter, Date Filters, Before`

A sanity check asks a question whose answer is known before looking. When the file disagrees with what is known, the file has a problem.

Today: one cell fails on purpose, and there are three Predict questions.

```
book = raw.copy()   # every fix is made to book
for column in ['as_of_date', 'maturity', 'price_date']:
    book[column] = pd.to_datetime(book[column],
        format='%Y-%m-%d', errors='coerce')
    print(book[column].isna().sum())   # count the failures
```

What the sanity checks found

Rows sent twice: removed

4 rows are exact copies of earlier rows. They carry 6,500,000 dollars of face and 6,950,710 of market value: the two differences from the cover note, to the dollar.

```
book.duplicated().sum()   # 4
book = book.drop_duplicates()
```

In Excel

Data, Remove Duplicates

```
=COUNTIF($B$2:$B$205, B2) > 1
```

Untidy tickers: tidied

8 tickers have spaces or lower case, so 121 tickers stand for 115 issuers. Grouped as received, THAA is 2,659,800 dollars. Tidied, 4,517,617.5. Tickers are invented.

```
book['ticker'] = (book['ticker']
    .str.strip().str.upper())
```

In Excel

```
=UPPER(TRIM(C2))
```

```
=NOT(EXACT(C2, UPPER(TRIM(C2))))
```

Stale prices: flagged, not changed

6 bonds carry a price dated in August, 35 to 57 days before the as-of date: 13,342,347.5 dollars, 2.72 percent of the book. The file cannot supply the right price.

```
stale = book[book['price_date']
             < book['as_of_date']]
```

In Excel

```
=COUNTIF(J2:J201, "<" & DATE(2026, 9, 30))
```

returns 6. Or Date Filters, Before

Headline numbers	Rows	Market value	Mean OAS	Mean yield
As received	204	497,769,925	187.83 bp	6.051%
Copies removed	200	490,819,215	185.75 bp	6.028%

Clean: CUSIP length, sector names, six rules for impossible values, market value against price. Left for the next step: 5 missing ratings.

Missing values: four things to do with a gap

Choice	Code	Rows	Grade table adds to	What it costs
Leave	`nothing`	200	475,476,677.5	grouped tables come up 15,342,537.5 short, silently
Drop	`dropna(subset=['rating'])`	195	475,476,677.5	five real bonds leave the book
Fill with a guess	`fillna('BBB-')`	200	490,819,215	five invented ratings nobody can tell from real ones
Fill with a label	`fillna('Not rated')`	200	490,819,215	one more row in every grouped table

5

ratings missing, in one column

3.13%

of the book's market value

9.35

mean() of score, over 195 rows

```
book['grade'] = book['grade'].fillna('Not rated')
print(book['score'].count(), len(book))   # 195 200
```

Decision 4: keep the five bonds and label them Not rated

The same in Excel

`=COUNTBLANK(F2:F201)` returns 5

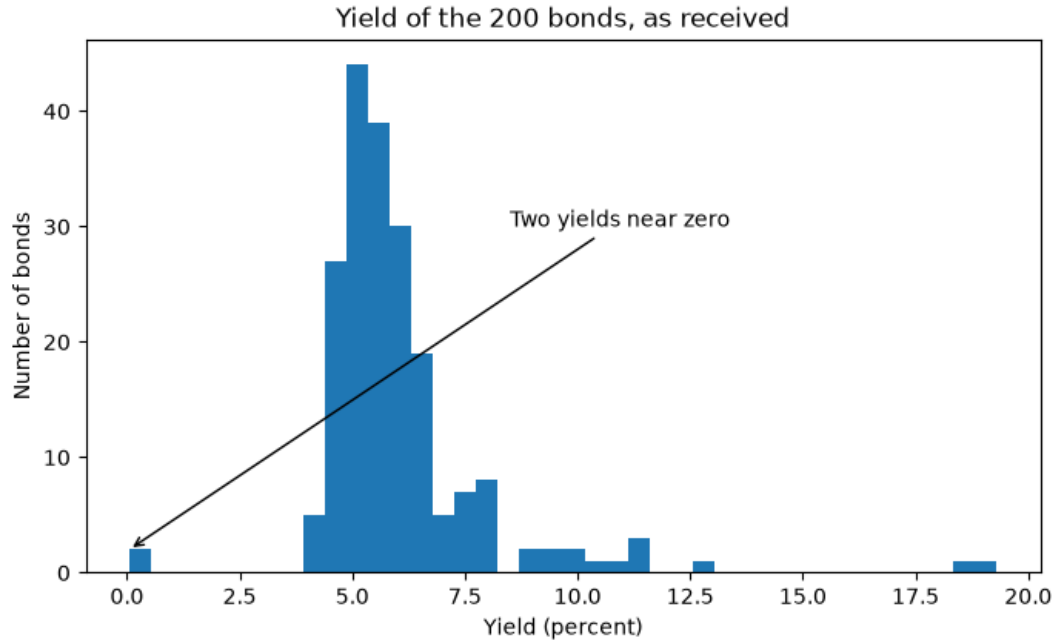
`=SUMIF(F2:F201, "", S2:S201)` returns 15,342,537.5

A pivot table with grade in Rows shows the empty cells as their own row, labelled (blank). SUMIF on a named grade leaves them out, as groupby() does.

To fill them with a label: select the column, F5, Special, Blanks, type the label, Ctrl+Enter.

=AVERAGE(T2:T201) leaves empty cells out, as mean() does.

One column at a time: the range and the shape




```
ax.hist(book['yield_pct'], bins=40)
book[columns].agg(['mean', 'median', 'skew']).T
```

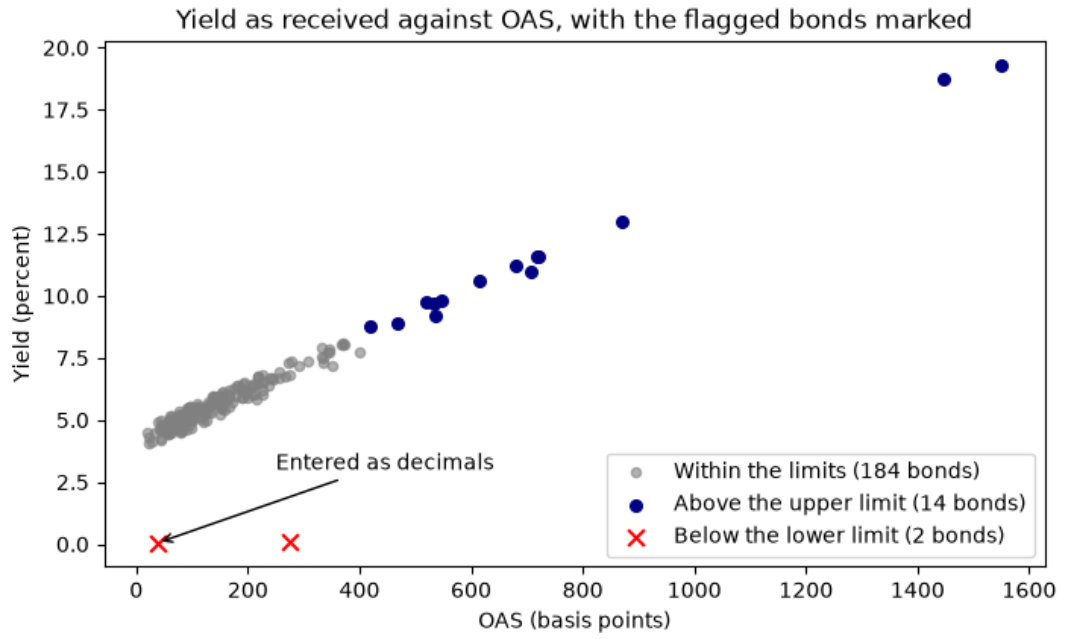
Mean, median, and skew, with the yields as received

Column	Mean	Median	Skew
price	98.83	99.42	-0.71
yield_pct	6.03	5.53	3.07
oas	185.75	135.50	3.93
duration	6.47	6.31	0.50

The histogram: a crowd from 4 to about 8 percent, a thin tail out to 19, and one short bar by itself at zero. It holds two bonds, at 0.04706 and 0.07308 percent. The next lowest yield is 4.089. The mean sits on the same side of the median as the long tail.

In Excel	Result
=SMALL(L2:L201, 1)	0.04706
=AVERAGE(M2:M201) and =MEDIAN(M2:M201)	185.75 and 135.5
=SKEW(M2:M201)	3.93

Outliers: an error, or a real bond?




```
upper_limit = q3 + 1.5 * iqr   # 8.401 percent
lower_limit = q1 - 1.5 * iqr   # 3.038 percent
base = book['yield_pct'] - book['oas'] / 100
```

16

yields flagged by the IQR rule

2

errors, entered as decimals

14

real bonds, all B+ or lower

The 2 errors: corrected

At 96.623 with a 7 percent coupon, a yield of 0.07308 is impossible. Yield less OAS: -0.333 and -2.677, against 3.691 to 4.585 for the other 198. Times 100 they are 4.706 and 7.308, and the mean yield goes from 6.028 to 6.088.

The 14 wide bonds: kept

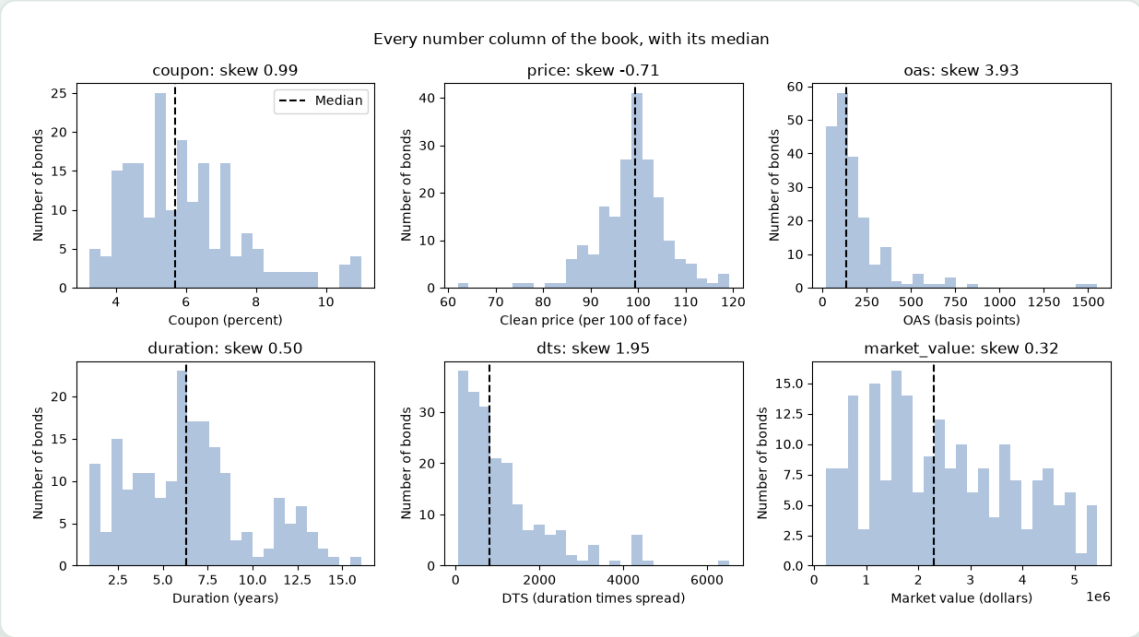
Every OAS is 418 bp or more, and all 14 sit on the line. Deleting them would cut the mean OAS from 185.75 to 144.21.

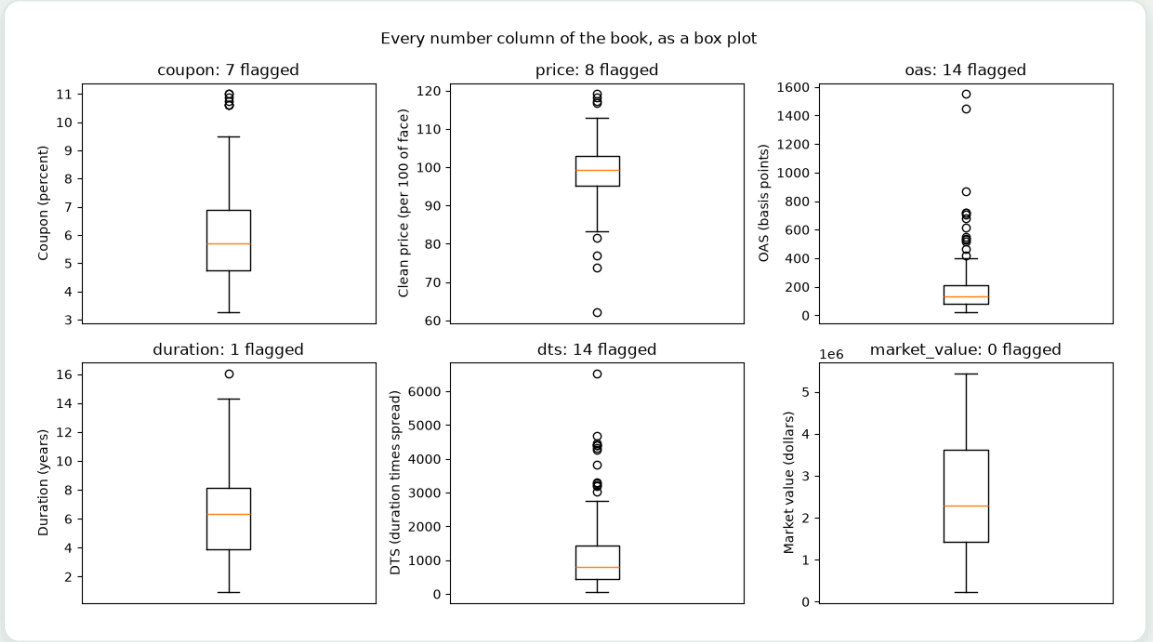
A row is never deleted because it is far away.

`=QUARTILE.INC(L2:L201, 3)` returns 6.39, and 5.049 with a 1

`=COUNTIF(L2:L201, "<3.038")` returns 2, on the yields as received

Every number column in one figure

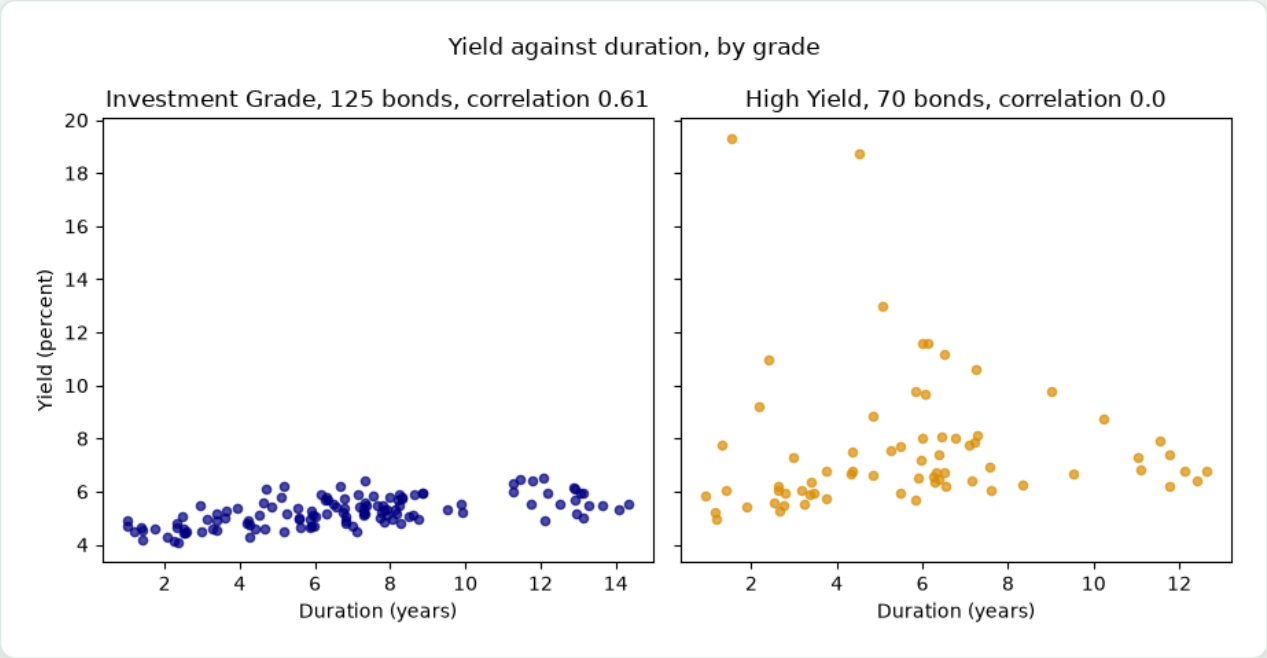





```
for ax, (column, label) in zip(axes.flatten(),
                               labels.items()):
    ax.hist(book[column], bins=25)
    ax.axvline(book[column].median())
```

One loop drew each grid, and a seventh column is one more line in the dictionary. OAS and DTS lean right, price leans left. Yield and OAS flag the same 14 bonds, DTS shares 10 of them, and the market value flags none.

Two columns at a time: the book and each grade




```
book['duration'].corr(book['yield_pct'])   # 0.01
part = book[book['grade'] == grade]
part['duration'].corr(part['yield_pct'])
```

Correlation of duration and yield

0.01	0.61	0.0
all 200 bonds	125 Investment Grade bonds	70 High Yield bonds

Near zero for the book does not mean unrelated: the two grades differ. Yield and OAS correlate at 0.99, and at 0.95 with the two yields as received. Two rows out of 200 moved it by 0.04.

The same in Excel

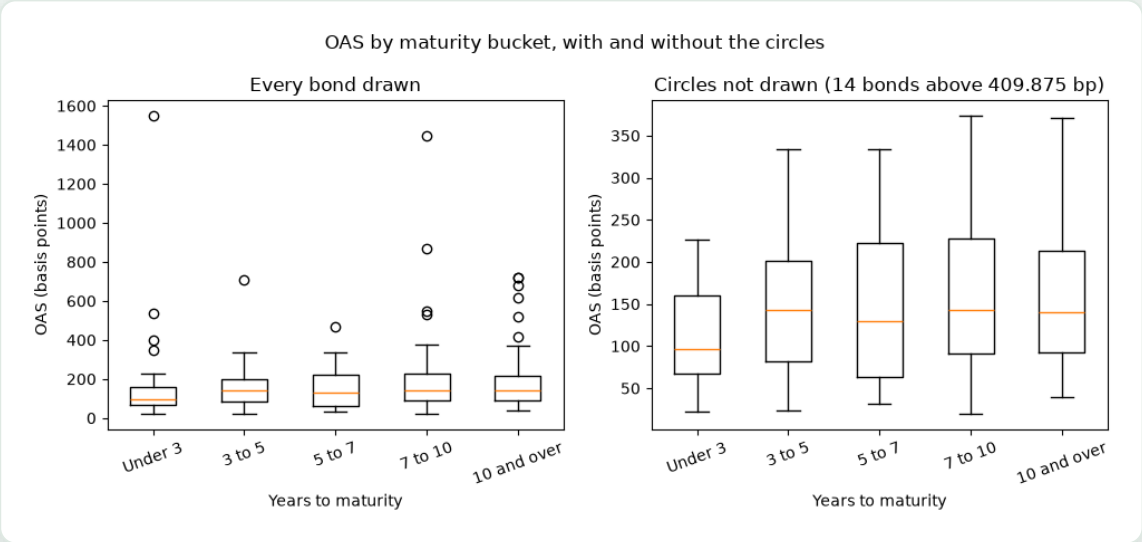
`=CORREL(L2:L201, M2:M201)` returns 0.99

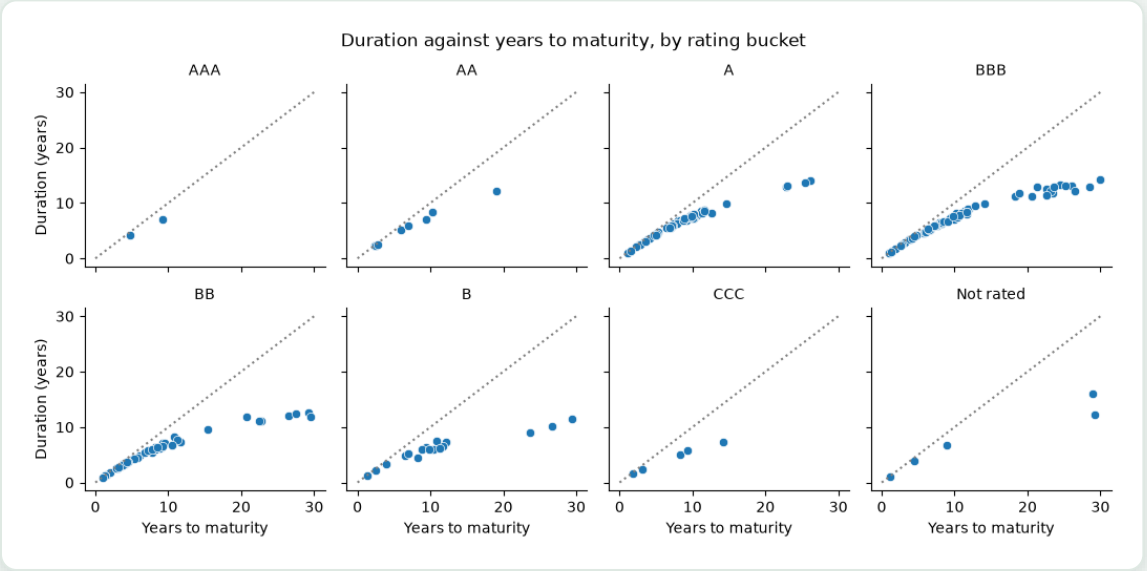
```
=CORREL(FILTER(N2:N201, V2:V201 = "Investment Grade"),
FILTER(L2:L201, V2:V201 = "Investment Grade"))
```

returns 0.61. FILTER is in Excel 365 and Excel 2021.

Hiding rows with Data, Filter does not change what CORREL returns.

A column built from dates






```
years = (maturity - as_of_date).dt.days / 365.25
pd.cut(years, bins=[0, 3, 5, 7, 10, 40],
       labels=[...], right=False)
```

Five maturity buckets hold 28, 28, 23, 51, and 70 bonds. The OAS does not rise with maturity (correlation 0.01). No bond has a duration above its years to maturity: every point is on or below the dotted line.

What the desk reports

#	Finding	How it was found	Bonds	What was done	Effect
1	Rows sent twice	row count, duplicated()	4	removed the copies	market value down 6,950,710 dollars
2	Untidy tickers	nunique(), tidy comparison	8	tidied	121 tickers became 115
3	Prices dated in August	date arithmetic	6	flagged, not changed	2.72 percent of the book
4	Ratings missing	isna().sum()	5	labelled Not rated	3.13 percent of the book, as its own row
5	Yields entered as decimals	IQR rule, yield less OAS	2	multiplied by 100	mean yield 6.028 to 6.088 percent
6	Very wide spreads	IQR rule, z-scores	14	kept: real bonds	none: they carry 41.54 bp of the mean OAS

Headline numbers	As received	As corrected
Rows	204	200
Market value, dollars	497,769,925	490,819,215
Mean OAS, basis points	187.83	185.75
Mean yield, percent	6.051	6.088

All three control totals match, with a difference of zero.

The test: the clean file, holdings.csv

17 of 19 columns match in every row, including the 8 tidied tickers and the 2 corrected yields. Two differ: rating in 5 rows and price_date in 6, the two that were reported and not fixed.

```
(book[column] != clean[column]).sum()   # rows that differ
```

Open: prices for 6 bonds, ratings for 5, and the reason 4 rows came twice

Make it repeatable: one function, and checks that stop

The day's decisions as one function

```
def clean_holdings(raw, ratings):
    book = raw.copy()
    # decisions 1 to 6, in order, one comment each
    return book
rebuilt = clean_holdings(raw, ratings)
bonds = len(rebuilt)
assert bonds == expected_bonds, 'message'
```

204

rows in, and 200 rows out

True

the same table as the day built

3

control totals, each an assert

The same check on the file as received stops

```
AssertionError: 204 bonds, and the cover note says 200
```

The decision log, built by code from the day's results

problem	rows	decision
Rows sent twice	4	removed the copies
Tickers with spaces or lower case	8	tidied
Prices dated before the as-of date	6	flagged, not changed
Ratings missing	5	kept, labelled Not rated
Yields entered as decimals	2	multiplied by 100
Very wide spreads	14	kept: real bonds

One workbook, two sheets, and the same in Excel

pd.ExcelWriter writes the sheet holdings, 200 rows and 22 columns, and the sheet decisions, 6 rows and 3 columns. Read back, the market value is 490,819,215.00 dollars.

The same steps next month: a Power Query query, then Refresh.

```
=IF(COUNTA(B2:B201) = 200, "ok", "CHECK: row count")
```

A check cell shows its result and does not stop anything.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Check a file against its control totals: the row count and the sums
- Find repeated rows, untidy text, dates held as text, and impossible values
- Count missing values, and say what leaving, dropping, and filling each do
- Flag outliers, and use a second column to tell an error from a real bond
- State every fix as a decision, and package them as one function with asserts

Week 3, Day 4 Case Study: Treasury Yields. The same order of work, on yields by tenor and year